

目 录

目 录	i
第 1 章 绪论	1
1.1 研究背景	1
1.2 研究意义	1
1.3 国内外研究现状	1
1.3.1 视频质量评价研究脉络	2
1.3.2 多模态融合与互动文本	2
1.4 研究内容与技术路线	2
1.4.1 研究内容	2
1.4.2 技术路线	3
1.5 论证脉络与章节安排	3
1.5.1 各章导读	3
第 2 章 相关技术与 MVQA 模型	5
2.1 前后端分离与系统开发技术栈	5
2.2 后端技术栈与服务端设计	5
2.2.1 框架、ASGI 与跨域	5
2.2.2 数据访问、状态机与分页	6
2.2.3 JWT 认证与流媒体安全	6
2.2.4 用户配置、敏感词与 MVQA 上下文	6
2.2.5 后台任务链与外部依赖	7
2.2.6 Bilibili 链接接入与下载链路	7
2.3 前端技术栈与交互实现	7
2.4 多模态视频质量分析 (MVQA) 模型	8
2.4.1 低秩多模态融合 (LMF) 实现要点	8
2.5 MVQA 推理实现与配置细节	11
2.5.1 推理流程与预处理	11
2.5.2 编码、融合与时序读出 (实现视角)	12

2.5.3	TemporalAggregation 与视频级读出	12
2.5.4	预处理与时间对齐实现要点	13
2.5.5	模型参数与配置管理	13
第 3 章	系统需求与总体设计	15
3.1	系统需求分析	15
3.1.1	功能性需求	15
3.1.2	非功能性需求	15
3.1.3	典型用例与交互流程	16
3.2	系统总体架构设计	16
3.2.1	设计原则（与架构决策对应）	16
3.2.2	B/S 架构与前后端分离模式	16
3.2.3	部署拓扑与运行假设	16
3.2.4	技术架构分层	16
3.3	数据库设计	17
3.3.1	设计原则与存储选型	18
3.3.2	概念结构设计（E-R 图）	18
3.3.3	逻辑结构设计（数据表结构）	18
3.4	系统功能模块设计	20
3.4.1	用户认证与权限管理模块	20
3.4.2	视频接入与预处理模块	20
3.4.3	多模态质量分析核心模块	20
3.4.4	结果可视化与配置管理模块	21
3.5	本章小结	21
第 4 章	系统实现与测试验证	23
4.1	开发环境与项目结构	23
4.1.1	开发与运行环境	23
4.1.2	启动方式与仓库目录	23
4.1.3	目录与配置路径约定	23
4.2	后端服务实现（FastAPI）	24
4.2.1	RESTful API 与 OpenAPI 契约	24
4.2.2	异步任务处理机制	24
4.2.3	JWT 认证与授权实现	24

4.2.4	业务服务层实现	26
4.2.5	扫码登录与会话边界	27
4.2.6	异常处理与失败恢复	27
4.3	前端界面实现 (Vue 3)	27
4.3.1	前后端联调与代理	27
4.4	测试与验证	28
4.4.1	测试环境、工具与 Web-MVQA 契约	28
4.4.2	功能测试 (模块级与业务流程)	28
4.4.3	接口、性能与推理一致性	29
4.4.4	性能与资源占用观察 (建议性)	30
第 5 章	总结与展望	31
5.1	全文总结	31
5.2	创新点	31
5.3	不足与未来展望	31
	参考文献	33
	致 谢	35

基于多模态融合的短视频质量评价系统的设计与实现

作者: 雷志刚 指导教师: 刘晓文 教授
(海南师范大学人工智能学院, 海口 571158)

摘要: UGC 短视频里, 观感好坏差别很大, 但不少做法还停留在“只看某一种信号”或“只给一个总分”, 很难把时间轴上哪一段出了问题说清楚。本文正是针对这一点, 搭了一套多模态视频质量评价的 Web 系统: 用 ViT、AST、RoBERTa 做特征, 经 LMF 和 TemporalAggregation, 同时给出片段级和视频级各 12 维得分; 后端用 FastAPI, 并在同进程里用 BackgroundTasks 做异步调度, 支持本地上传、从 B 站链接拉取以及长时间推理, 前端用 Vue 3 做播放和图表; 数据存在 SQLite, 登录会话走 JWT (JWT_SECRET), 推理时的参数通过 contextvars 和用户个人设置对齐。在已经部署好的权重下, 可以稳定复现随时间对齐的曲线和雷达图; 若要对公开数据做大规模主观评测, 则作为后续工作另行开展。

关键词: 视频质量评价; 多模态融合; 低秩多模态融合; 弹幕分析; 片段级预测; Transformer 编码器; 前后端分离; FastAPI; Vue.js

Design and Implementation of a Short-Video Quality Assessment System Based on Multimodal Fusion

Author: Zhigang Lei Supervisor: Prof. Xiaowen Liu
(School of Artificial Intelligence,
Hainan Normal University, Haikou 571158, P. R. China)

Abstract: In UGC short videos, the quality varies greatly, but many evaluation methods still rely on “examining only a single feature” or “assigning a single overall score,” making it difficult to pinpoint exactly which segment of the timeline has issues. This paper addresses this issue by developing a web-based system for multimodal video quality evaluation: it uses ViT, AST, and RoBERTa to extract features, followed by LMF and TemporalAggregation, to simultaneously provide 12-dimensional scores at both the clip and video levels; the backend uses FastAPI, with asynchronous scheduling via BackgroundTasks within the same process, supporting local uploads, fetching from Bilibili links, and long-duration inference; the frontend uses Vue 3 for playback and visualization; data is stored in SQLite, with login sessions managed via JWT (JWT_SECRET).

Inference parameters are aligned using `contextvars` and user preferences. Under pre-deployed weights, the system reliably reproduces time-aligned curves and radar charts; conducting large-scale subjective evaluations on public datasets will be addressed as future work.

Keywords: video quality assessment; multimodal fusion; danmaku analysis; clip-level prediction; low-rank multimodal fusion; Transformer encoders; FastAPI; Vue 3

第 1 章 绪论

1.1 研究背景

移动端与推荐算法推动下，短视频消费已高度普及；与此同时 UGC 供给在清晰度、剪辑与互动密度上差异显著。平台日更量级下，单靠人工抽检无法覆盖，而流水线里常见的客观 IQA 或违规词过滤^[1-2]往往不承接弹幕、口播等互动信号，难以把“看得是否舒服”与“评论区是否吵”放在同一套标尺下讨论。Transformer 族模型为视听文联合表征提供了工程上可复用的骨干^[3-4]；在中文语境里，按时间轴对齐的弹幕可视为群体即时反馈，与画面、口播一并支撑沿时间切分的质量描述，因而需要把多模态融合与可运维的 Web 交付结合起来讨论。

从课程设计与毕设验收角度，仅展示离线 Notebook 难以证明“可部署、可配置、可多人使用”；本课题因此将推理服务、用户配置、媒资管理与可视化置于同一 B/S 闭环中，使评委能够通过浏览器操作复现分析结果，而不仅依赖静态截图。

1.2 研究意义

理论意义 在模型这一侧，本文用片段级输出把「质量随时间怎么起伏」说清楚，并在同一向量空间里把弹幕和视听特征合在一起看；工程上还可以借着张量形状、配置是否对得上，检查多模态前向路径有没有一路跑通、没有断档。模态消融这类更严格的对比实验没有放进正文，等有更合适的数据集和算力再补做。

实践意义 最后交付的是一个打开浏览器就能用的分析台：运营可以按多维得分和片段曲线做批量筛查，创作者则能把「哪一段不对劲」对齐到具体画面和弹幕；两边共用同一套推理服务和同一套前端可视化，不用各维护一套脚本、越改越乱。

1.3 国内外研究现状

文献中的 VQA 路线已从手工特征与 CNN/时空建模过渡到深度感知与跨模态交互^[5-6]；面向 UGC 时，弹幕、口播与画面、梅尔谱之间的时间对齐以及“从论文实验到可安装、可配置服务”之间的落差仍常被提及。中文语境下预训练语言模型与工程化实践可参考国内专著^[7]，以降低弹幕与口播文本表征与部署沟通的门槛；但把 LMF 级融合、片段回归头与可登录、可改 num_clips 的站点放在同一条仓库主线里，在公开讨论中仍相对少见。本课题以片段级 LMF 加 B/S 实现直接对准这一空档。

1.3.1 视频质量评价研究脉络

从有参考的全参考、到只看失真画面的无参考，再到如今的 UGC，质量评价这条线总体上是：大家越来越关心「和人主观打分是否对得上」，而不是只盯着「和源视频有多像」^[2]。无参考那一类方法^[1]往流水线里嵌比较方便，但要想单独把弹幕节奏、口播清不清楚这类和叙事相关的细项说清楚，它往往不够用。深度模型普及之后，单模态特征可以端到端学出来；而 UGC 在分辨率、码率、剪辑风格上又特别杂，所以比起只给一个全局分，把时间轴切成片段、一段一段给结论，更容易把问题钉到具体时段。

1.3.2 多模态融合与互动文本

跨模态注意力与双塔结构在图文、音视任务中已较为成熟^[4]；弹幕作为中文社区特有的时间序列文本，与画面并置时须解决分轨对齐与缺失（无弹幕）两种情形。工程上，若仅交付离线脚本，实验配置与线上配置往往脱节；本系统将用户级 Setting、推理入口 `predict_video_qa` 与前端展示绑定在同一配置语义下，便于答辩演示与可复现部署。

1.4 研究内容与技术路线

1.4.1 研究内容

(1) 多模态特征与融合。编码前端用 ViT^[8]、AST^[9]、RoBERTa^[10]（负责弹幕与旁白），需要时再加一条可选的 Whisper 旁路^[11]；片段级和视频级两条分支都走 LMF，先把各模态投影到同一维度，再做三模态的低秩张量式交互，细节见第二章。

(2) 片段级与视频级联合预测。时间轴会切成 T_c 个互相对齐的片段；默认配置里 `num_clips=125`，模型的 `max_clips` 和同样长度的 `clip_pos_embedding` 要和 checkpoint 对上号。用户在设置页改 `num_clips` 时，若和当初导出权重时的 T_c 不一致，就容易加载失败或推理时张量维度对不齐。输出包括每片段 12 维波形和整段视频 12 维向量；视频级分支在进 LMF 之前，对音、视的片段编码分别做 TemporalAggregation 读出，旁白仅参与全局融合。

(3) 全栈系统。后端用 FastAPI + SQLAlchemy（默认 SQLite），通过 `schedule_video_analysis` 配合同进程 BackgroundTasks，把单条/批量上传、从 URL 拉取、reanalyze、弹幕解析一直到 MVQA 推理串成一条链；视频列表支持服务端筛选、排序（前端默认按创建时间从新到旧，并可按状态、文件名过滤）。前端用 Vue 3 和 ECharts 做登录、播放、波形/雷达、缩略图列表和配置。敏感词带管理员审批，管理员 UID 支持合并配置，与第三章里的需求条目一一对应。

1.4.2 技术路线

本文重心为在已给定或已导出的 **MVQA 权重** 前提下，完成预处理—推理—落库—可视化的全栈集成与可复现部署；顺序为：媒资与配置准备 → 推理链路 与 B/S 实现（FastAPI + Vue 3 与 SQLite，含异步调度与 JWT）→ 契约测试与可选 Nginx 部署。训练数据构造、离线训练细节及 PLCC/SRCC 等客观指标评测不作为正文展开，与摘要及第 5 章展望一致。见图 1.1。

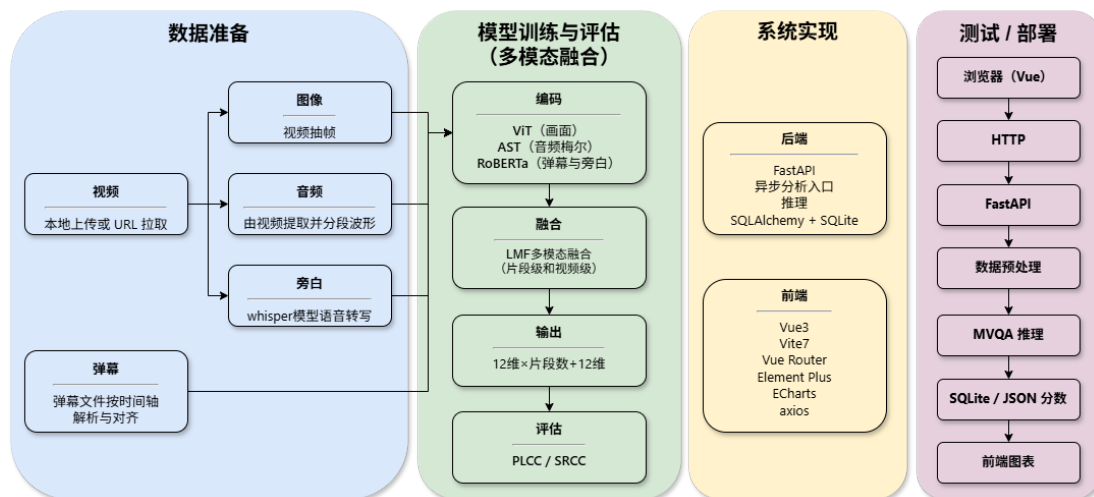


图 1.1 系统分析流程概览

1.5 论证脉络与章节安排

全文按问题 → 共同语言 → 需求与数据 → 实现与验证 → 收束展开：第 1 章提出问题与落点；第 2 章给出 B/S、REST/JWT、异步调度（BackgroundTasks）、MVQA 机理与 predict_video_qa 工程细节（表 2.1、图 2.1、图 2.2、图 2.3、表 2.2、表 2.3）；第 3 章将目标操作化为 FR/NFR、库表与模块映射（表 3.1、图 3.1、图 3.2、表 3.5）；第 4 章对照源码落地并做契约测试（图 4.1、式 (4.4.1)、式 (4.4.2)、式 (4.4.3)）；第 5 章归纳创新点与展望。

1.5.1 各章导读

第 2 章在同一章内串联接口契约与模型前向，读者可先浏览表 2.1 建立“能调哪些 API”的印象，再对照图 2.3 理解张量从预处理到双分支输出的路径，避免“接口章与模型章符号不一致”。第 3 章用 FR 编号贯穿需求—实现映射，数据库与状态机公式为第 4 章测试断言提供依据。第 4 章侧重可核对性：后台任务与会话边界、分页不变式、Web 与离线推理的数值容差；不写无关接口罗列。第 5 章收束创新点并诚实讨论数据集、工程局限及未开展的训练侧评测。全文默认假定权重已就绪，与上文技术路线收窄一致。

第 2 章 相关技术与 MVQA 模型

本章按「工程栈 → REST 接口 → MVQA 在干什么 → 推理和配置怎么落地」这条线来写：前半段先把 B/S、FastAPI/Vue 和接口表说清楚；后半段把 MVQA 的公式、数据流、`predict_video_qa` 里张量形状和配置要注意什么放在同一章里写完，免得「讲理论的太短、讲算法的又飘在外面」，也方便第 3 章写需求、第 4 章对实现时大家用同一套符号。

2.1 前后端分离与系统开发技术栈

系统是典型的 B/S、前后端拆开，靠 REST 传 JSON、multipart 或流；整体怎么走见图 2.1。分析往往要跑很久，所以上传/入库接口先快速把响应返回，再由 `schedule_video_analysis` 通过 `BackgroundTasks.add_task(process_video_task, ...)` 把真正的分析丢到后台跑。开发时 `vite.config.js` 会把 `/api` 和 `/uploads` 代理到后端（清单 4.4）。

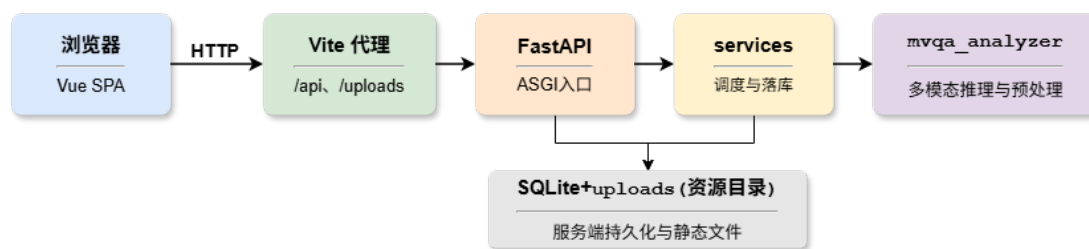


图 2.1 B/S 主线数据流

2.2 后端技术栈与服务端设计

2.2.1 框架、ASGI 与跨域

入口在 `main.py`: `app = FastAPI(...)` 交给 Uvicorn 按 ASGI 跑，路由里同步、异步写法都能接。跨域交给 `CORSMiddleware`，白名单读 `settings.py` 里的 `CORS_ORIGINS`（也可用环境变量，逗号分隔；不设时默认 `["*"]`）。若允许来源写成 `*`，浏览器不允许带 cookie 凭证，所以 `allow_credentials` 会是 `false`；上线时一般改成具体前端域名并开 HTTPS。

FastAPI 用 Pydantic v2 校验返回体：`VideoResponse`、`VideoListResponse` 等在 `schemas.py`，响应模型里嵌套的 `class Config` 设 `from_attributes = True`，和 SQLAlchemy ORM 对象直接对上，少手写一层字典拼来拼去。

2.2.2 数据访问、状态机与分页

数据库默认用 SQLite (`check_same_thread=False`)。Video.status 怎么变见图 2.2; `progress/progress_stage` 用来标走到哪一步 (解码、视觉、融合等), 方便前端轮询。

列表接口 GET `/api/videos` 要满足分页约定: `items`、`total`、`skip`、`limit` 成套返回; 前端做无限滚动时让 `skip` 跟着已加载条数往后挪, 一改筛选条件就把 `skip` 归零。

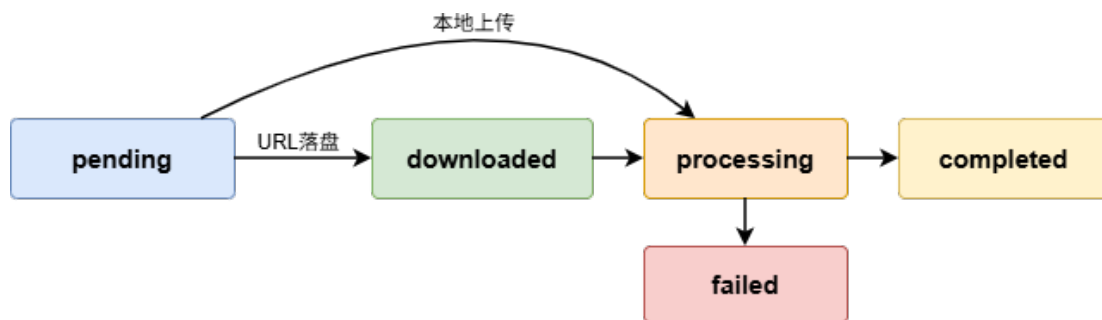


图 2.2 视频状态典型迁移

2.2.3 JWT 认证与流媒体安全

登录态用无状态 JWT (HS256): 载荷里有 `sub`、`exp`, 服务端验签并对时钟。普通 JSON 请求走 `Authorization: Bearer`; 但 `<video>` 的 `src` 不好自定义 Header, 所以流媒体 `/api/videos/{id}/stream` 额外支持 `?token=`, 解码逻辑和 Bearer 那套一致。Range 请求回 206, 每块大约 8 KiB。上传路径要过 `is_safe_path` 并落在真实 `uploads` 目录里; 文件名会换成 UUID。

2.2.4 用户配置、敏感词与 MVQA 上下文

`ConfigManager` 会按 `user_id` 读写多行 `Setting` (键名包括 `video_processing`、`model_paths`、`download_settings` 等), 同时把全局 `SensitiveWord` 表收成一张扁平列表 `sensitive_words`, 一并写进返回给调用方的字典里。用户第一次还没有配置时, 就先回落到 `DEFAULT_CONFIG` 并落库保存。换句话说, 服务端把「每个用户自己的 JSON 配置」和「全局敏感词」整理成一份推理前就能拿到的字典, 后面的路由和 `mvqa_analyzer` 共用这一份。

分析阶段里, `mvqa_analyzer` 借助 `contextvars` (`MVQA_CONFIG_USER_ID`) 在异步、层层嵌套的调用链中带着当前用户 id, 这样 `Config.get_config()` 才能按人读出各自的 `num_clips`、采样帧数、模型路径等——也就是把「Web 里这次是谁」和「推理用哪套参数」对齐到同一个人身上, 不必去动全局环境变量。

2.2.5 后台任务链与外部依赖

`process_video_file` 入库之后交给 `schedule_video_analysis` (见第 2.1 节) ; `predict_video_qa` 的结果经 `analyze_video_quality` 写成 `video_score/clip_scores/suggestions`, 推理阶段受 `_INFERENCE_LOCK` 约束, 避免多路任务同时抢推理资源。依赖见 `requirements.txt` (`fastapi`、`torch`、`transformers` 等)。

2.2.6 Bilibili 链接接入与下载链路

鉴权过了之后, `dispatch_download` 按域名挑下载器; B 站就调 `bilidown.download_video`, 清晰度和要不要弹幕看用户 `download_settings`。文件落到 `uploads`, 路径写回 `Video`。扫码拿到的 `Cookie`、`sessdata` 只在服务端用, 不和下载逻辑绑死在一处。

```

1 # backend/download_registry.py (excerpt)
2 from __future__ import annotations
3
4 from typing import Any, Callable, Dict
5 from urllib.parse import urlparse
6
7 from backend.bilidown import download_video as bili_download_video
8
9 Downloader = Callable[..., Any]
10
11 REGISTRY: Dict[str, Downloader] = {
12     "bilibili.com": bili_download_video,
13     "b23.tv": bili_download_video,
14 }
15
16
17 def _hosts_for(url: str) -> str:
18     host = (urlparse(url).hostname or "").lower()
19     return host
20
21
22 def dispatch_download(url: str, output_dir: str, **kwargs) -> None:
23     host = _hosts_for(url)
24     for suffix, fn in REGISTRY.items():
25         if host == suffix or host.endswith("." + suffix):
26             fn(url, output_dir, **kwargs)
27             return
28     # 默认尝试 B 站解析 (历史行为: 含 BV 的 URL 由 bilidown 内部抛错)
29     bili_download_video(url, output_dir, **kwargs)

```

代码 2.1 `download_registry.py` 中按域名调度 `dispatch_download`

2.3 前端技术栈与交互实现

前端是 `Vue 3 + Vite 7 + Vue Router + Element Plus`; 页面状态主要在 `Home.vue` 里用 `ref/reactive` 管。`frontend/src/api/index.js` 里配好 `axios` 根地址、自动带 `Bearer`, 遇到 401 做清理。接口汇总见表 2.1。

方法	路径 (前缀 /api)	说明
GET	/health	探活
POST	/videos/upload、/videos/upload/batch、/videos/upload/url	本地上传、批量、B 站 URL
GET/DELETE	/videos 、/videos/id 、/videos/id/stream	分页列表、详情、Range 流 (?token=)
POST	/videos/id/reanalyze	重新分析
GET/PUT	/config	用户 Setting
GET/POST/PUT/DELETE	/sensitive-words、/sensitive-words/requests*	敏感词与申请审批
GET/PUT	/admin/bilibili-mids	管理员 MID
GET	/auth/qr-code、/auth/qr-status/key 、/auth/me	取二维码、轮询状态 (路径参数名 qr_key)、当前用户
POST	/auth/logout、/auth/bilibili/refresh	登出、校验 B 站 sessdata 是否仍有效

表 2.1 核心 REST 接口归类

2.4 多模态视频质量分析 (MVQA) 模型

光看画面或光听声音都不够，还得把互动文本一起算进来；本模型用 Transformer 编码器抽特征，经 LMF 和时序读出，得到按片段和整段视频各一套 12 维输出 (OUTPUT_DIM)。实现以 mvqa.py、mvqa_analyzer.py 为准：fusion_dim=512，max_clips 和 clip_pos_embedding 要和权重对上；推理时从 model_paths.traced_model_path 装载。

长视频先切成 T_c 片 (num_clips)，每片有梅尔谱、帧、对齐弹幕，旁白则整段编码。片段级把 $(\mathbf{f}^a, \mathbf{f}^d, \mathbf{f}^v)$ 送进 LMF，加上位置嵌入，再过 clip_regressor；视频级先把 $\{\mathbf{f}_i^a\}$ 、 $\{\mathbf{f}_i^v\}$ 用 TemporalAggregation 压成序列表示，再和 $\mathbf{f}^s$ 一起做 LMF 和 video_regressor。两路输出同一套 12 维语义，方便和表 2.2 对表。

$$\begin{aligned}\mathcal{Y}_{\text{clip},i} &= g_{\text{clip}}(\text{LMF}_{\text{clip}}(\mathbf{f}_i^a, \mathbf{f}_i^d, \mathbf{f}_i^v) + \mathbf{e}_i), \\ \mathbf{y}_{\text{vid}} &= g_{\text{vid}}(\text{LMF}_{\text{vid}}(\bar{\mathbf{f}}^a, \mathbf{f}^s, \bar{\mathbf{f}}^v)).\end{aligned}\quad (2.4.1)$$

g_{clip} 、 g_{vid} 是 MLP；编码器长什么样、融合怎么接，见图 2.3；LMF 参见 [12]，RoBERTa 参见 [10,7]。若要算 loss，见 MVQA.compute_loss。

2.4.1 低秩多模态融合 (LMF) 实现要点

LowRankMultimodalFusion 先把三模态投影到 d_f ，再按秩 lmf_rank 做低秩分解式的交互：片段级、视频级各一套，后面分别接 clip_regressor、video_regressor，片级还加 clip_pos_embedding。换权重时要保证 lmf_rank、维度和位置表长度一致。代码摘录见清单 2.2。

```

1 # backend/model/fusion.py (excerpt) — LowRankMultimodalFusion
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5 from torch.nn.parameter import Parameter
6
7
8 class LowRankMultimodalFusion(nn.Module):
9     def __init__(self, audio_dim, text_dim, visual_dim,
10                 fusion_dim=512, rank=4, dropout=0.1):
11         super().__init__()
12         self.rank = rank
13         self.fusion_dim = fusion_dim
14         self.proj_audio = nn.Sequential(
15             nn.Linear(audio_dim, fusion_dim),
16             nn.LayerNorm(fusion_dim),
17             nn.ReLU()
18         )
19         self.proj_text = nn.Sequential(
20             nn.Linear(text_dim, fusion_dim),
21             nn.LayerNorm(fusion_dim),
22             nn.ReLU()
23         )
24         self.proj_visual = nn.Sequential(
25             nn.Linear(visual_dim, fusion_dim),
26             nn.LayerNorm(fusion_dim),
27             nn.ReLU()
28         )
29         self.audio_factor = Parameter(torch.Tensor(rank, fusion_dim + 1, fusion_dim))
30         self.text_factor = Parameter(torch.Tensor(rank, fusion_dim + 1, fusion_dim))
31         self.visual_factor = Parameter(torch.Tensor(rank, fusion_dim + 1, fusion_dim))
32         self.fusion_weights = Parameter(torch.Tensor(1, rank))
33         self.fusion_bias = Parameter(torch.Tensor(1, fusion_dim))
34         self.dropout = nn.Dropout(dropout)
35         self._init_weights()
36
37     def _init_weights(self):
38         nn.init.xavier_normal_(self.audio_factor)
39         nn.init.xavier_normal_(self.text_factor)
40         nn.init.xavier_normal_(self.visual_factor)
41         nn.init.xavier_normal_(self.fusion_weights)
42         nn.init.zeros_(self.fusion_bias)
43
44     def forward(self, audio_emb, text_emb, visual_emb):
45         batch_size = audio_emb.size(0)
46         a = self.proj_audio(audio_emb)
47         t = self.proj_text(text_emb)
48         v = self.proj_visual(visual_emb)
49         a_one = torch.ones(batch_size, 1, device=a.device)
50         t_one = torch.ones(batch_size, 1, device=t.device)
51         v_one = torch.ones(batch_size, 1, device=v.device)
52         _a = torch.cat([a_one, a], dim=-1)
53         _t = torch.cat([t_one, t], dim=-1)
54         _v = torch.cat([v_one, v], dim=-1)
55         fusion_a = torch.einsum('bi,rio->bro', _a, self.audio_factor)
56         fusion_t = torch.einsum('bi,rio->bro', _t, self.text_factor)
57         fusion_v = torch.einsum('bi,rio->bro', _v, self.visual_factor)
58         fusion_zy = fusion_a * fusion_t * fusion_v
59         output = torch.einsum(
60             'br,brd->bd',
61             self.fusion_weights.expand(fusion_zy.shape[0], -1),
62             fusion_zy
63         )
64         output = output + self.fusion_bias
65         return self.dropout(output)

```

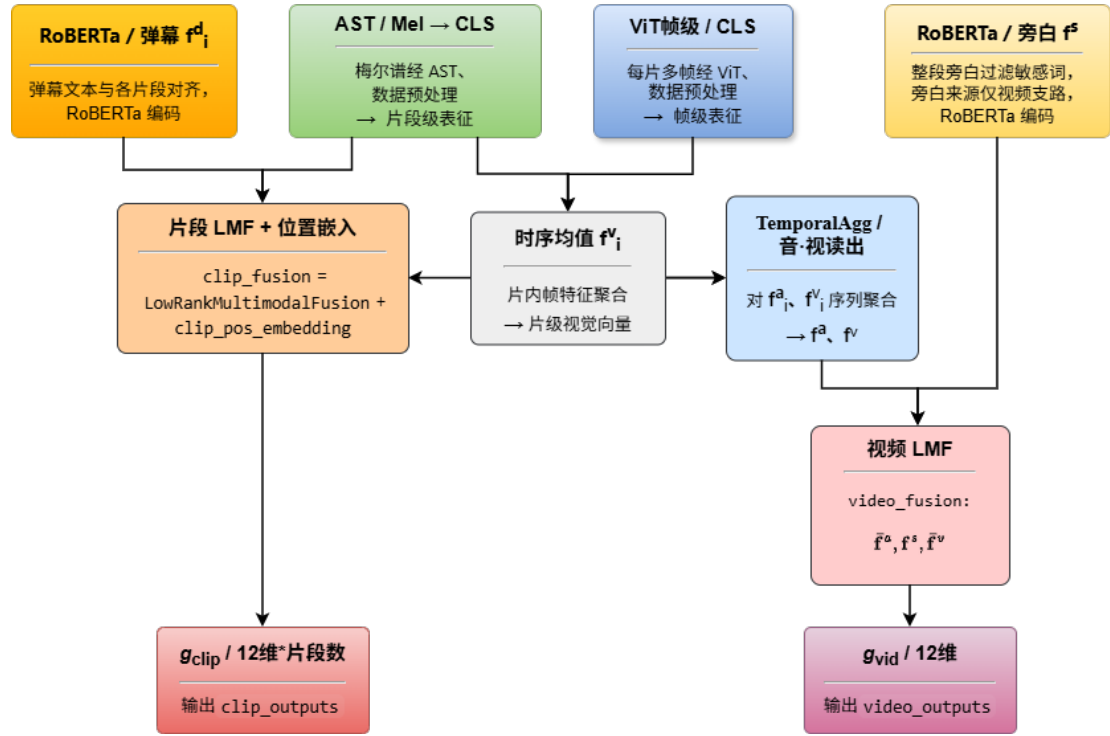
代码 2.2 model/fusion.py 中 LowRankMultimodalFusion: 投影、低秩因子与 einsum 融合

```

1 # backend/model/mvqa.py — MVQA.__init__ core wiring (excerpt; encoders omitted)
2 from torch import nn
3 from backend.model.fusion import LowRankMultimodalFusion, TemporalAggregation
4
5
6 class MVQA(nn.Module):
7     def __init__(
8         self,
9         fusion_dim=512,
10         output_dim=12,
11         lmf_rank=4,
12         temporal_aggregation="attention",
13         dropout=0.3,
14         max_clips=125,
15     ):
16         super().__init__()
17         audio_dim = text_dim = visual_dim = 768 # from encoders in full class
18         self.clip_pos_embedding = nn.Embedding(max_clips, fusion_dim)
19         self.clip_fusion = LowRankMultimodalFusion(
20             audio_dim, text_dim, visual_dim, fusion_dim, rank=lmf_rank, dropout=dropout
21         )
22         self.clip_regressor = nn.Sequential(
23             nn.Linear(fusion_dim, fusion_dim * 2),
24             nn.ReLU(),
25             nn.Dropout(dropout),
26             nn.Linear(fusion_dim * 2, fusion_dim),
27             nn.ReLU(),
28             nn.Dropout(dropout),
29             nn.Linear(fusion_dim, output_dim),
30         )
31         self.audio_temporal_agg = TemporalAggregation(
32             audio_dim, aggregation_type=temporal_aggregation, dropout=dropout
33         )
34         self.visual_temporal_agg = TemporalAggregation(
35             visual_dim, aggregation_type=temporal_aggregation, dropout=dropout
36         )
37         self.video_fusion = LowRankMultimodalFusion(
38             audio_dim, text_dim, visual_dim, fusion_dim, rank=lmf_rank, dropout=dropout
39         )
40         self.video_regressor = nn.Sequential(
41             nn.Linear(fusion_dim, fusion_dim),
42             nn.ReLU(),
43             nn.Dropout(dropout),
44             nn.Linear(fusion_dim, output_dim),
45         )

```

代码 2.3 model/mvqa.py 中 MVQA: clip_fusion/video_fusion 与 TemporalAggregation 衔接



工程与模型契约（承上启下）。表 2.1、图 2.3、式 (2.4.1) 后面写需求、做验收都按这三样来：第 3 章写功能时要能指到接口和 MVQA 数据流；第 4 章落地和测试时，路径、张量形状、时间对齐要和这里一致。

2.5 MVQA 推理实现与配置细节

推理链路本来可以单独成章，这里仍放在同一章里顺着 MVQA 机理往下写，读起来顺，也方便对着源码查。

2.5.1 推理流程与预处理

整条链从 `predict_video_qa` 进：先把片子准备好（抽帧、梅尔谱、弹幕/旁白）→ 交给 `MVQAPreprocessor` → 再进 `MVQA.forward` → 最后 `postprocess_predictions` 收个尾。出来是两路，片段张量 $\mathbf{Y}_{\text{clip}} \in \mathbb{R}^{B \times T_c \times d_o}$ 和视频向量 $\mathbf{y}_{\text{vid}} \in \mathbb{R}^{B \times d_o}$ ；和图 2.3 画的是一回事，代码主体在 `_predict_video_qa_impl`。

预处理这块，`VideoFrameExtractor` 和 `VideoAudioSegmentProcessor` 共用同一时间轴，把长视频切成 T_c 个窗：一边出帧张量，一边出梅尔谱；弹幕走 `ASSParser`，再由 `FineGrainedAligner` 对齐成「每片一句」喂给文本侧，旁白则交给 `AsideFilter` 整段转写、过滤一遍。都齐了以后，`MVQAPreprocessor` 里用 ViT/AST 抽视听特征、RoBERTa 做分词，在 `batch` 维打包成 `MVQA.forward` 要的输入；某片没有弹幕也别空着，用空串把 T_c 行填满，免得文本维对不齐。更细的

```

1 # backend/mvqa_analyzer.py (excerpt)
2 import contextvars
3 from typing import Optional
4
5 MVQA_CONFIG_USER_ID: contextvars.ContextVar[Optional[int]] = contextvars.ContextVar(
6     "mvqa_config_user_id", default=None
7 )
8
9
10 def predict_video_qa(
11     video_path: str,
12     ass_path: str = "",
13     device: str = "cpu",
14     *,
15     user_id: Optional[int] = None,
16 ):
17     ctx_token = None
18     if user_id is not None:
19         ctx_token = MVQA_CONFIG_USER_ID.set(user_id)
20
21     try:
22         return _predict_video_qa_impl(video_path, ass_path, device)
23     finally:
24         if ctx_token is not None:
25             MVQA_CONFIG_USER_ID.reset(ctx_token)
26
27
28 def _predict_video_qa_impl(video_path: str, ass_path: str = "", device: str = "cpu"):
29     ...

```

代码 2.4 mvqa_analyzer.py 中 MVQA_CONFIG_USER_ID 与 predict_video_qa 用户配置绑定

符号与前文 MVQA 小节一致，这里不重复展开。

索引	语义（后处理与建议规则见 mvqa_analyzer）
0-4	画面子维度（清晰/色彩/饱和/稳定/亮度等）
5-9	音量子维度
10	综合分 [0, 100]
11	弹幕/互动相关（无弹幕时可截断）

表 2.2 12 维输出索引语义

2.5.2 编码、融合与时序读出（实现视角）

张量怎么走和式 (2.4.1)、图 2.3 一致：MVQA.forward 在 batch 维并行编码，旁白只进视频级那条 LMF；出来后再走 postprocess_predictions 做后处理，12 维各代表什么见表 2.2。

2.5.3 TemporalAggregation 与视频级读出

$\{f_i^a\}$ 、 $\{f_i^v\}$ 先经 TemporalAggregation 池化成 $\bar{f}^a$ 、 $\bar{f}^v$ ，再和 f^s 一起进视频级 LMF；num_heads 和编码维要整除，见表 2.3。联调时先盯紧 T_c 、 d_f 和权重是否一致。

2.5.4 预处理与时间对齐实现要点

画面和声音要对齐，关键在「同一时间格子上切」：抽帧、梅尔谱共用一套时间网格——`frames_per_clip` 决定每片能出多少个视觉 token，`target_size` 跟 ViT 预训练时吃进去的尺寸一致；梅尔谱在 `audio_processing.sample_rate` 下开的窗，也要和片的起止对齐，不能各算各的。弹幕这条线：ASSParser 先把时间轴上的字幕读出来，FineGrainedAligner 再决定多条合并还是挑一句代表，填进该片 RoBERTa；哪怕这一片一条弹幕都没有，也照样用空串占一行，免得 T_c 维文本张量中间缺行。旁白可以接 Whisper 转写（可选），过 AsideFilter 后整段进 RoBERTa，只走视频级、不进片级，和图 2.3 里旁白只接到视频 LMF 一致。

2.5.5 模型参数与配置管理

配置键从哪儿来、怎么落到推理里，前文 `DEFAULT_CONFIG`、`ConfigManager`、`mvqa_analyzer.Config` 已经串过一遍；表 2.3 把和 MVQA 张量形状「绑在一起、一改就全得改」的项单独摘出来，联调时对着表核对最省事。

键 / 字段	典型值	含义
<code>video_processing.num_clips</code>	125	T_c ：与 <code>clip_pos_embedding</code> 行数、抽帧数一致
<code>video_processing.frames_per_clip</code>	5	每片帧数 T_f
<code>video_processing.target_size</code>	[224,224]	ViT 输入空间尺寸
<code>audio_processing.sample_rate</code>	16000	分轨/梅尔谱采样率
<code>model_paths.*</code>	见 <code>DEFAULT_CONFIG</code>	预训练目录与 <code>traced_model_path</code>
<code>Config.FUSION_DIM</code>	512	LMF 投影维 d_f
<code>Config.OUTPUT_DIM</code>	12	d_o
<code>MVQA(max_clips=)</code>	125	位置表长度 $\geq T_c$
<code>TemporalAggregation(num_heads=)</code>	8	须整除各模态编码维 d_a 或 d_v （默认 768）
<code>MVQAPreprocessor.process_text(max_length=)</code>	512	分词截断长度

表 2.3 Setting / `mvqa_analyzer.Config` 摘录：与 MVQA 形状强相关的项

若要在这套结构上接着训，可以用 `MVQA.compute_loss`（`joint` 模式下片段/视频 MSE，还带片段方差和片段—视频辅助项，权重在源码里）；没有单独的 `mvqa_loss.py`。

权重加载与部署 `_predict_video_qa_impl` 优先用 `Config.get_traced_model_path()`：路径在就 `torch.load(..., weights_only=False)` 取出 `state_dict` 再 `load_state_dict`；不行再试 `torch.jit.load`；路径都没有时模型保持随机初始化（仅方便把结构跑

通，分数没有实际意义)。加载成功后 `model.to(device)`，在 `torch.no_grad()` 里前向。

工程稳定性(日志与IO) `main.py` 里给 Uvicorn、Transformers 等加了日志过滤器，少刷 WebSocket 断开、权重复用那类提示；删盘文件时有限次重试，缓解 Windows 上句柄占着不放的情况。机器吃紧时可以把 `num_clips`、`frames_per_clip` 调小，或换更小的 Whisper，在精度和耗时之间折中。

与后续章节的关系 张量形状和配置以表 2.3、表 2.2 为准；第 4 章实现与测试要和这里对齐，后面式 (4.4.3) 一类核对才说得通。

第3章 系统需求与总体设计

本章将第1、2章的研究目标操作化为FR/NFR、库表与模块边界（3.1–3.4）；每项需求在表3.1中可追溯到源码锚点，供第4章实现与测试对照。

3.1 系统需求分析

用户角色含创作者、解说作者与运营辅助；功能与非功能约束分别以表3.1、表3.2为验收基准（与OpenAPI及第二章常量一致）。

3.1.1 功能性需求

要点见表3.1，定量参数同第二章代码常量。

编号	需求摘要	主要实现位置
FR-1	B站扫码、JWT、401统一处理	Login.vue、src/api/index.js、/api/auth/*
FR-2	本地/批量/URL入库、reanalyze、格式校验、覆盖更新	FileUpload.vue、InputDialog.vue、/api/videos/upload*、/api/videos/{id}/reanalyze
FR-3	异步MVQA、进度字段、用户级配置、score_change	job_queue.py、main.py (process_video_task)、services.py、mvqa_analyzer.py
FR-4	任务轮询、分页列表、状态/文件名筛选、删除	Home.vue、FileList.vue、/api/videos*
FR-5	流媒体、ECharts、雷达、弹幕层	ChartMain.vue、WidgetCard.vue、stream
FR-6	设置页、敏感词CRUD、申请审批、管理员MID、首启种子	Settings.vue、/api/config、/sensitive-words*、/api/admin/bilibili-mids
FR-7	帮助页、健康检查、静态挂载	Help.vue、/api/health、/uploads
FR-8	列表缩略图、thumbnail_path生命周期	main.py (write_thumbnail_jpeg)、FileList.vue

表3.1 核心功能性需求与实现映射

3.1.2 非功能性需求

表3.2中性能相关符号与第2章异步调度及第3.2节轮询叙述一致： T_{http} 为典型API请求响应时延， L 为分页列表单次拉取耗时， T_{poll} 为前端轮询间隔，

$T_{\text{infer}}(n_c, f_c)$ 为推理耗时（随片段数 n_c 、每片帧数 f_c 等变化）。

维度	要点
性能	T_{http} 、 L 、 T_{poll} 、异步调度、 $T_{\text{infer}}(n_c, f_c)$ 、 <code>_INFERENCE_LOCK</code>
安全	JWT、行级隔离、UUID、路径沙箱、TLS、CORS、日志脱敏

表 3.2 非功能属性摘要

3.1.3 典型用例与交互流程

主干流程见图 3.1；分支要点（B 站先落盘、`num_clips` 与权重一致、敏感词与 MID）已体现在表 3.1。



图 3.1 典型交互

3.2 系统总体架构设计

本节给出 B/S 职责切分与三层逻辑划分；模块级锚点见表 3.5。主线为 SQLite + 同进程异步任务 + 本地 uploads；docker-compose.yml 可作为容器化编排参考。

3.2.1 设计原则（与架构决策对应）

单一入口、信任边界在服务端、推理与 HTTP 解耦、schemas/axios 契约显式化。

3.2.2 B/S 架构与前后端分离模式

浏览器侧 SPA + localStorage + 播放/ECharts；服务端鉴权、媒资、推理与配置。API 前缀 /api，静态媒资 /uploads；开发态 Vite 代理，生产常 Nginx 反代 dist。首包时延不含后台推理；感知完成约为 $T_{\text{first}} + T_{\text{infer}} + \varepsilon$ ($T_{\text{poll}} \approx 2\text{s}$)。

3.2.3 部署拓扑与运行假设

BackgroundTasks 与 API 同进程调度分析任务。`_INFERENCE_LOCK` 保证单机 GPU 单路推理。Nginx 可拆分端口与 TLS；库可换 PostgreSQL 等；容器部署须持久化 uploads 与库文件并配置 GPU。

3.2.4 技术架构分层

逻辑三层与仓库对应关系见图 3.2；mvqa.py 中 LMF/TemporalAggregation 与 mvqa_analyzer.Config 同构，预处理见 *_utils.py。



图 3.2 逻辑三层与仓库对应关系

3.3 数据库设计

在第二章与 3.2 节已明确 B/S 持久化与架构语境、行级隔离策略之后，本节先给出 ORM 映射、E-R 与逻辑表结构，再于 3.4 节按流水线展开功能模块，使“存什么”与“业务如何编排”顺序衔接。

系统持久层采用 SQLAlchemy ORM 映射至 SQLite 单文件库（默认 `sqlite:///./video_rating.db`，相对后端进程工作目录），连接参数 `check_same_thread=False` 以适配 FastAPI 多协程环境下的会话复用；引擎在连接建立时执行 `PRAGMA journal_mode=WAL`，减轻并发读写下写锁等待。表结构以用户维度隔离业务数据；敏感词落在全局表 `sensitive_words`，与具体用户无外键，与“Setting 行 + 内存默认字典”共同构成文本过滤词源。

评分类 JSON: `video_score` 为 $d_o=12$ 维向量，`clip_scores` 为 $n_c \times d_o$ 嵌套列表（ n_c 来自 `num_clips`），`suggestions` 为派生文本列表。

3.3.1 设计原则与存储选型

媒资与元数据分离 大块头的东西——视频、弹幕文件——放在磁盘上（例如 uploads/），库里只记路径、时间、状态、评分这类元数据，不把整文件塞进 BLOB，既省库压力，也方便边下边播、以后要备份迁移也好办。

初始化与模式演进 服务启动时会先跑 `Base.metadata.create_all(engine)`，缺的表就补上；第一次启动且敏感词表还是空的，就用 `DEFAULT_CONFIG` 塞一批种子数据。毕设里没有上 Alembic 那种迁移工具，实现章里可以交代一句：小步迭代时这样省事，真要上生产再考虑升级脚本怎么走。

行级安全边界 查 `Video`、`Setting` 这类表都带上 `user_id`，别串号；播流媒体时拿 token 解出来，也要和这条资源是谁的对得上。

3.3.2 概念结构设计（E-R 图）

谁和谁一对多、基数怎么定，直接看图 3.3；`Setting` 里各命名空间长什么样、默认 JSON 骨架，对照表 3.3。图里没单独写出来的，这里补两句：（1）`User.email` 只是为满足 ORM 非空而填的占位，不做真实邮箱校验；（2）走 `ConfigManager.save_config` 时如果一并带上 `sensitive_words`，有可能整表重写全局敏感词，要和审批流怎么放行对齐好，别和后台直接改表那条路打架。

视频任务状态（概念层） 状态能怎么跳、`VideoStatus` 取哪些值，式 (3.3.1) 里概括了；以后若要把弹幕多版本都留档，可以再加实体例如 `DanmuAsset`，眼下先用 `danmu_path` 一个字段顶着。

键	典型 value 字段（节选）
<code>video_processing</code>	<code>num_clips</code> , <code>frames_per_clip</code> , <code>target_size</code>
<code>audio_processing</code>	<code>sample_rate</code> （如 16000）
<code>model_paths</code>	<code>whisper_model</code> , <code>audio_model</code> , <code>text_model</code> , <code>visual_model</code> , <code>traced_model_path</code>
<code>download_settings</code>	<code>video_quality</code> , <code>default_video_quality</code> , <code>audio_quality</code> , <code>download_danmaku</code> 等

表 3.3 `Setting.key` 命名空间与默认配置呼应（值列为 JSON 骨架）

新用户第一次登录可以按表 3.3 把各命名空间种子化一份；敏感词仍走全局表，不往 `Setting` 的键行里塞。

3.3.3 逻辑结构设计（数据表结构）

逻辑表的主键、外键与字段节选见图 3.4；`videos` 列级说明见表 3.4。

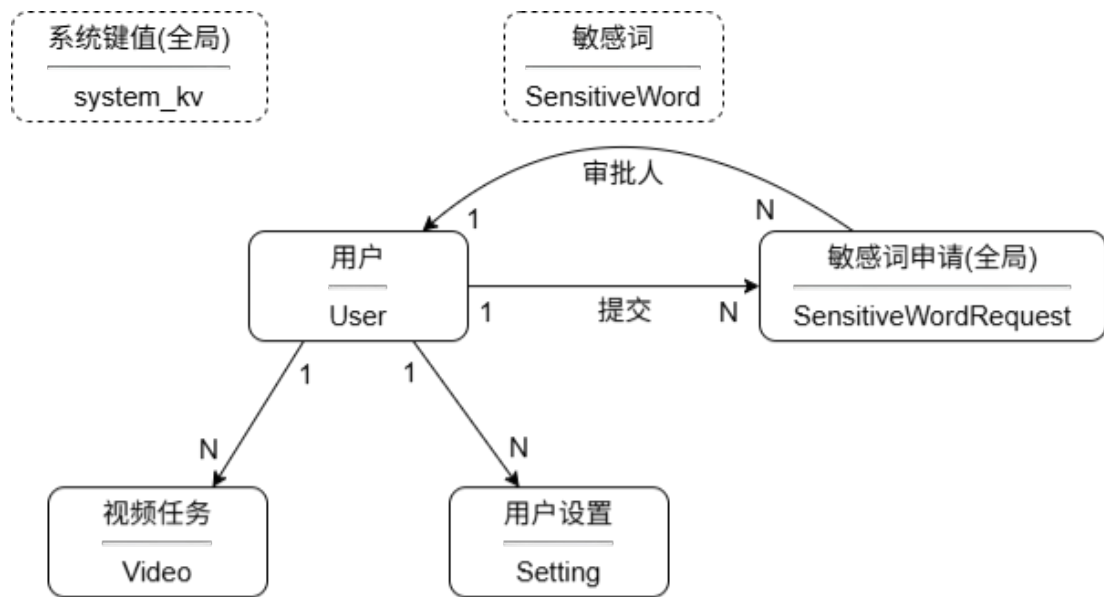


图 3.3 概念层主要实体及联系

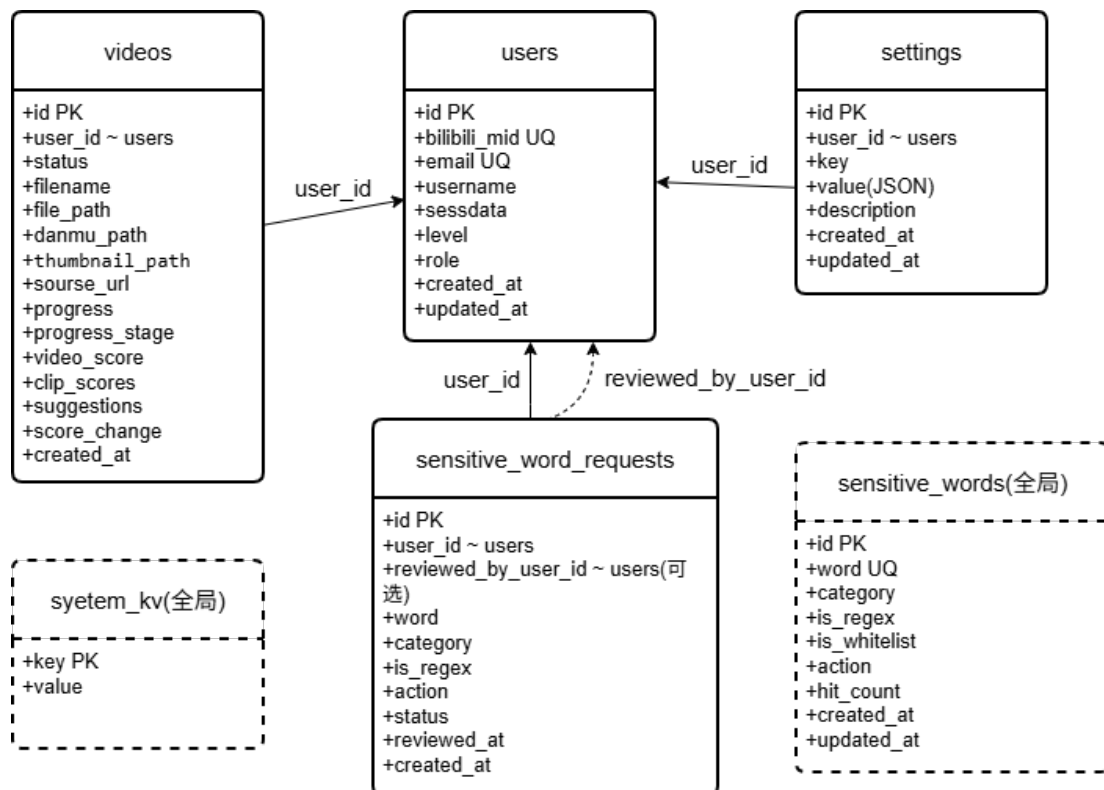


图 3.4 逻辑表结构概览

针对 videos 表的 status 字段，取值域约束如下：

$$\text{status} \in \{\text{pending}, \text{downloaded}, \text{processing}, \text{completed}, \text{failed}\}. \quad (3.3.1)$$

字段名	类型	约束	说明
id	INTEGER	PK	自增主键
filename	VARCHAR	INDEX	展示用文件名
file_path	VARCHAR		本地路径或 s3:// 等对象存储键
danmu_path	VARCHAR	NULL	弹幕 ASS/XML 路径
thumbnail_path	VARCHAR	NULL	列表缩略图 JPEG，存 uploads/ 下
source_url	VARCHAR	NULL	URL 下载时的原始链接
progress	FLOAT	默认 0	分析进度 0-100
progress_stage	VARCHAR	NULL	可读阶段标签（如解码、视觉、融合）
status	VARCHAR	默认 pending	式 (3.3.1)
video_score	JSON	NULL	d_o 维列表；索引 0-4 画面、5-9 音、10 综合、11 弹幕相关（同表 2.2）
clip_scores	JSON	NULL	$n_c \times d_o$ 嵌套；前端折线均匀时间近似
suggestions	JSON	NULL	generate_suggestions 生成
score_change	FLOAT	NULL	覆盖上传时相对旧综分的变化量
created_at	DATETIME	默认当前时间	创建/入队时间
user_id	INTEGER	FK $\rightarrow$ users.id, NULL	归属用户

表 3.4 逻辑表 videos（视频任务与评分结果）

其余业务表与图 3.4 一致；对外 API 不暴露 sessdata。

3.4 系统功能模块设计

认证—接入—MVQA—可视化四块与源码锚点见表 3.5；数据流仍对齐第二章图 2.3。以下分节标题仅作导航，细节以表为准。

3.4.1 用户认证与权限管理模块

源码锚点见表 3.5 行 3.4.1；补充说明：401 时清理本地令牌，与第二章 src/api/index.js 拦截器一致，避免半登录态误请求。

3.4.2 视频接入与预处理模块

见表 3.5 行 3.4.2；须保证入库媒资进入 MVQA 前张量形状与 num_clips 及第二章工具链一致。

3.4.3 多模态质量分析核心模块

见表 3.5 行 3.4.3；除得分落库外须维护失败终态与 score_change，12 维语义见表 2.2。

3.4.4 结果可视化与配置管理模块

见表 3.5 行 3.4.4；时间片 $t_i \approx (i + \frac{1}{2})T/n_c$ ，片段曲线与雷达图绑定同一套 video_score/clip_scores，避免展示与库内快照漂移。

模块（3.4 节）	主要源码锚点
3.4.1 认证与权限	bililogin.py、main.py（JWT、require_admin）、admin_mids_store.py、Login.vue、Header.vue、models.User/SystemKV
3.4.2 接入与预处理	download_registry.py、bilidown.py、main.py（上传/URL/任务）、mvqa_analyzer.MVQAPreprocessor、video_utils.py、audio_utils.py、text_utils.py（AsideFilter、ASSParser、FineGrainedAligner）
3.4.3 分析核心	mvqa_analyzer.predict_video_qa、mvqa.py、services.analyze_video_quality（建议与列表规范化）
3.4.4 可视化与配置	Home.vue、ControlPanel.vue、ChartMain.vue、WidgetCard.vue、DataBlock.vue、Settings.vue、Help.vue、/api/config、/api/sensitive-words*、/api/admin/bilibili-mids

表 3.5 功能模块与源码映射总览（扩展）

3.5 本章小结

本章完成 FR/NFR（表 3.1、表 3.2）、架构视图（图 3.1、图 3.2）、SQLite 与表 3.5。第 4 章衔接实现与测试。

第 4 章 系统实现与测试验证

本章顺着「先实现、再测」往下写：开发环境、仓库长什么样见表 4.1；异步任务怎么串见图 4.1。验收时对照表 3.1、表 3.5、表 2.1、表 4.4 就够串起一条线。

4.1 开发环境与项目结构

4.1.1 开发与运行环境

后端用 Python 3, 依赖都写在 `requirements.txt` 里, 主心骨是: `fastapi==0.104.1`、`uvicorn`、`sqlalchemy==2.0.23`、`pydantic`、`python-multipart`、`pyjwt`, 推理侧再拉上 `torch/torchaudio`、`transformers`、`moviepy`、`opencv-python-headless` 等。数据库默认单机 SQLite (`sqlite:///./video_rating.db`), 以后要换 MySQL/PostgreSQL 之类, 改连接串即可。

前端走 Node + Vite 7, 跑起来靠 `vue 3.5.x`、`vue-router 4.x`、`element-plus`、`axios`、`echarts 6.x`; `package.json` 里虽然装了 `pinia@3`, `main.js` 里暂时没挂全局 Store, 页面状态主要在 `setup` 里用 `ref/reactive`, 再配合 `localStorage`, 以后要用 `Pinia` 也有余地。

4.1.2 启动方式与仓库目录

开发时在后端工程目录跑 `uvicornmain:app--reload--host0.0.0.0--port8000` (`main:app` 按你实际包名改); `--reload` 改代码就热起来。`UPLOAD_DIR` 在 `main.py` 里按 `__file__` 定到同级的 `uploads` 绝对路径, 这样你从哪个目录起进程, 上传目录和 `is_safe_path` 的基准都不会飘。

前端开发跑 `npm run dev`; 上线前 `npm run build`, 再用 Nginx 把 `/api`、`/uploads` 反代过去。具体版本 `pin` 在 `requirements.txt` 和 `package.json` 里。

仓库按「前后端分目录」放, 表 4.1 把主要文件干啥用捋了一遍。

4.1.3 目录与配置路径约定

后端用 `__file__` 当锚点去解析 `uploads` 和默认 SQLite, 避免你从别的目录起服务时出现「文件明明传了却找不到」的乌龙。权重、HuggingFace 缓存一般在 `model_paths` 或环境变量里指好, 实验记录里最好写死绝对路径和版本, 后面按式 (4.4.3) 那种核对才对得上。前端打包出来只有静态文件, 别把 `.env` 里的密钥打进去; `VITE_*` 是构建期注入的, 上线时要单独配后端地址和 CORS。

路径/文件	职责
main.py、services.py、config.py	路由、JWT、后台任务、业务编排、配置
mvqa_analyzer.py、mvqa.py、fusion.py、*_utils.py	MVQA、预处理、B 站模块
models.py、database.py	ORM 与 SQLite
src/api/index.js、router.js、components/*.vue、vite.config.js	前端 API、路由、页面、代理

表 4.1 仓库主要文件与职责

4.2 后端服务实现 (FastAPI)

4.2.1 RESTful API 与 OpenAPI 契约

资源怎么划、第二章表 2.1 已经写过：接口前缀 /api，静态媒资 /uploads，拉流 /api/videos/{id}/stream?token=。这里不再抄一遍表；实现上动词按 REST 习惯来，返回体用 Pydantic（例如 VideoResponse）卡死，和前端 src/api/index.js 对得上号。

跨域、OpenAPI 与分页。 CORSMiddleware 开发时放宽点，上线建议收紧；/docs、/redoc 用来对契约。GET /api/videos 要求 $\text{skip} \geq 0$ 、 $1 \leq \text{limit} \leq 1000$ ，默认 limit 跟前端一页多少条对齐；返回 VideoListResponse，前端靠它算 hasMore（看 $|\text{items}| < \text{total}$ 是否还够翻）。

依赖注入与数据库会话。 get_db 用生成器 yield 出 SessionLocal，请求结束会话就关掉，符合 FastAPI 那套依赖生命周期；路由里别在全局长期攥着一个 Session。后台任务 process_video_task 要自己再开一个会话，因为请求里的 Session 早就释放了。

4.2.2 异步任务处理机制

入口是 schedule_video_analysis，里面 add_task(process_video_task, ...) 把活丢给后台。process_video_file 先返回，真正迁状态、写分在后台跑；数据库会话要自己建，别跟请求里那个 Session 混着用。整条链和图 4.1、清单 4.1、4.2 对得上。

4.2.3 JWT 认证与授权实现

实现走 HS256:create_access_token 签发，路由里 Depends(get_current_user) 取当前用户；播视频用 Query 里的 token，和 (video_id, user_id) 一起核对是

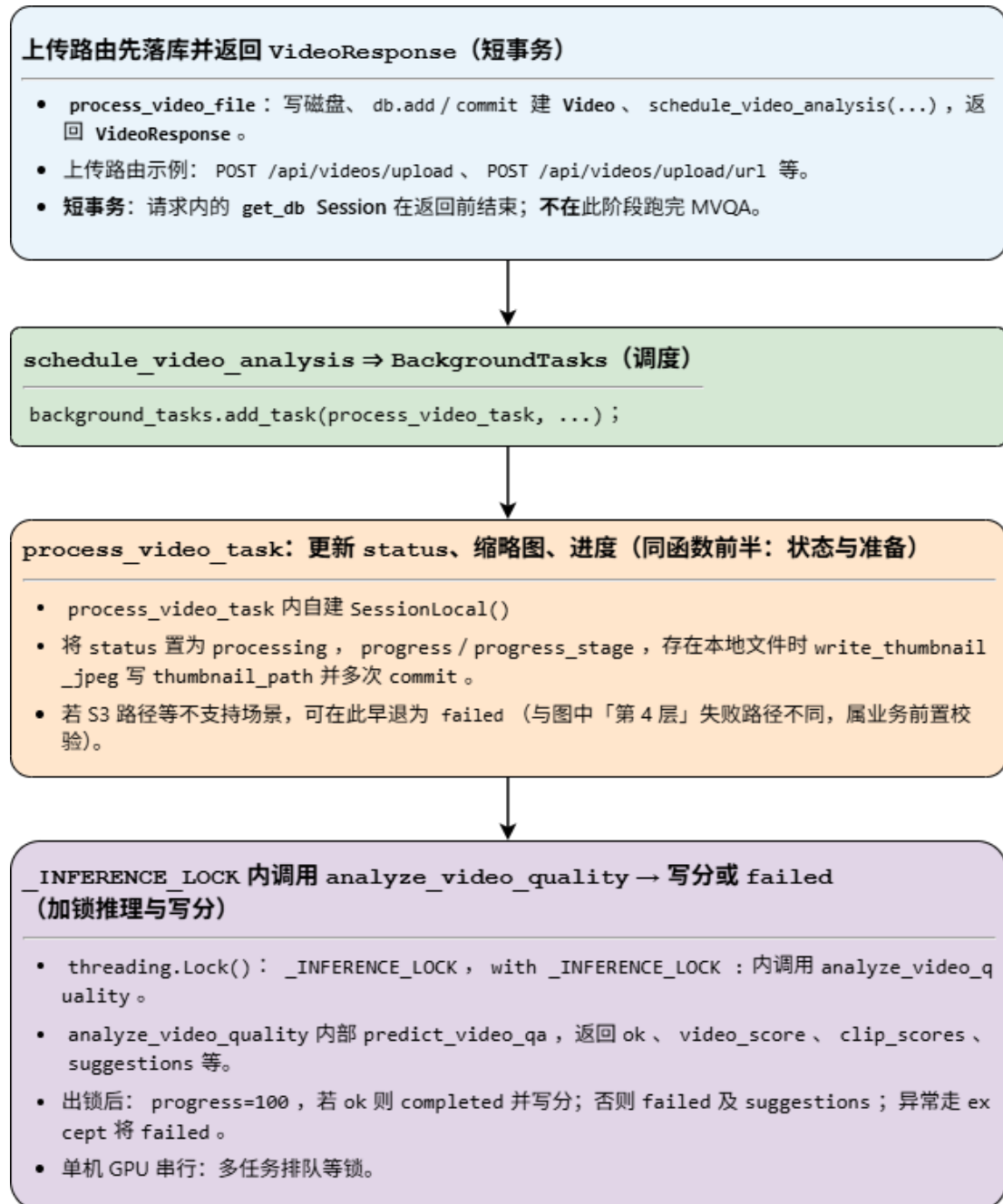


图 4.1 异步分析链与 HTTP 响应解耦

```

1 # backend/job_queue.py (excerpt)
2 from __future__ import annotations
3
4 from typing import Any, Optional
5
6
7 def schedule_video_analysis(
8     background_tasks: Any,
9     video_id: int,
10    file_path: str,
11    danmu_path: str = "",
12    old_score: Optional[float] = None,
13 ) -> None:
14     from backend.main import process_video_task
15
16     background_tasks.add_task(process_video_task, video_id, file_path, danmu_path,
17                               old_score)

```

代码 4.1 job_queue.py 中 schedule_video_analysis 经 BackgroundTasks 调度

```

1 # backend/main.py (excerpt)
2 import threading
3 from backend.services import analyze_video_quality
4
5 _INFERENCE_LOCK = threading.Lock()
6
7
8 def process_video_task(video_id: int, file_path: str, danmu_path: str = "", old_score=
9     None):
10     # ... SessionLocal, load Video, fp/dp paths, status=processing ...
11     with _INFERENCE_LOCK:
12         results = analyze_video_quality(fp, dp, user_id=video.user_id)
13     # ... write video_score / failed ...

```

代码 4.2 main.py 中 _INFERENCE_LOCK 与串行推理（节选骨架；process_video_task 全量见仓库）

不是本人，Range 请求回 206。JWT_SECRET 在 settings.py，上线务必够随机并全程 HTTPS。

4.2.4 业务服务层实现

analyze_video_quality 把 predict_video_qa 吐出来的张量整理成列表，再拼建议文案；没弹幕时会把尾维截掉，推理挂了则退回空分，别让任务一直卡在半空；以后要加缓存可以靠 get_file_hash 当键。调用长什么样见清单 4.3。

```

1 # backend/services.py (excerpt)
2 def analyze_video_quality(file_path: str, ass_path: str = "", user_id=None):
3     from backend.mvqa_analyzer import predict_video_qa
4
5     clip_outputs, video_outputs = predict_video_qa(
6         file_path, ass_path, user_id=user_id
7     )

```

代码 4.3 services.py 中 analyze_video_quality 调用 predict_video_qa

4.2.5 扫码登录与会话边界

`/api/auth/qr-code` 和 `/api/auth/qr-status/{qr_key}` 跟 `Login.vue` 轮询配合，扫完 B 站码就发 JWT，载荷里只有用户 id 和过期时间。普通接口用 `Depends(get_current_user)`；视频流那边浏览器不好塞 `Authorization`，所以用 Query token，解码跟 Bearer 一套逻辑，再核对 (`video_id`, `user_id`) 是不是你的。登出清服务端会话还是清 `localStorage` 看你怎么实现，关键是 401 之后前端别还拿废令牌去轮询。

4.2.6 异常处理与失败恢复

`process_video_task` 里要把推理异常兜住，把 `Video.status` 写成 `failed`，别永远停在 `processing`。删磁盘文件走 `safe_remove_file` 加有限次重试，缓解 Windows 句柄占着删不掉。权重没装好还硬跑，分数没意义，所以部署时要盯 `checkpoint` 路径，`/api/health` 也可以加探针看权重是否就绪。前端遇到 `failed` 要给看得懂的原因（后端有写的话），并允许 `reanalyze` 或重新传。

4.3 前端界面实现（Vue 3）

路由、页面谁干啥，跟表 3.1 对得上；大屏三列大概 2:0.8:1，时间轴按 $t_i \approx (i + \frac{1}{2})T/n_c$ 画。`ControlPanel`、`ChartMain` 等一起用，默认是按 `num_clips` 把总长 T 均匀切开；设置页点保存要和表 2.3 对齐，别和后台各说各话。

4.3.1 前后端联调与代理

`vite.config.js` 把 `/api`、`/uploads` 代理到后端（清单 4.4）；轮询间隔 T_{poll} 、`cleanupTimers` 等见表 4.2。

```

1 // frontend/vite.config.js (excerpt)
2 import { defineConfig, loadEnv } from 'vite'
3 import vue from '@vitejs/plugin-vue'
4
5 export default defineConfig(({ mode }) => {
6   const env = loadEnv(mode, process.cwd(), '')
7   const devApiTarget = env.VITE_DEV_API_TARGET || 'http://127.0.0.1:8000'
8
9   return {
10     plugins: [vue()],
11     server: {
12       proxy: {
13         '/api': { target: devApiTarget, changeOrigin: true },
14         '/uploads': { target: devApiTarget, changeOrigin: true },
15       },
16     },
17   }
18 })

```

代码 4.4 vite.config.js 开发态代理 `/api`、`/uploads`

项	含义
baseUrl	开发态为 /api; 设 VITE_BACKEND_ORIGIN 时为绝对后端根 +/api
timeout	60000 ms, 长视频上传较慢时可避免过早中断
Authorization	请求拦截器自动附加 Bearer + token
401 响应	清除 localStorage 并回首页或刷新, 防僵尸会话
videoStreamUrl	/ 分别处理需 token 的流与公开的静态弹幕路径
uploadsPublicUrl	

表 4.2 前端 src/api/index.js 与联调相关的关键参数

4.4 测试与验证

本节从环境、功能、推理是否对得上号三方面验收; 建议先对着 /docs 把契约捋顺, 再跑端到端场景。

需求—测试可追溯性 第 3 章表 3.1 里 FR-1–FR-8, 每条都该有「打算怎么测」: 表 4.4 里 T1–T5 覆盖鉴权、上传、拉流、覆盖上传和 num_clips 等, 方便写脚本; T6–T7 对应 FR-6–FR-8, 多半靠手工或清单打勾。非功能那一摊对照表 3.2。

4.4.1 测试环境、工具与 Web–MVQA 契约

基线环境: 单机 Python 3.x + requirements.txt, Node + Vite + Vue 3, SQLite 和 uploads 放一块盘; MVQA 走 PyTorch, 有 GPU 就开 CUDA。要做对比实验, shell 和 torch、transformers 版本先固定死。工具上 /api/health、/docs、浏览器 DevTools、curl 一起用; 写接口会吃 GPU、_INFERENCE_LOCK 和异步队列, 只读接口想压测可以 Locust/k6 试试。端到端要和 mvqa_analyzer._predict_video_qa_impl 里的张量形状咬死; 表 4.3 列的是 Web 侧 Setting 和 mvqa_analyzer.Config 要对齐的键。

项	验证意图
num_clips、frames_per_clip、target_size	与权重/trace 张量形状一致
FUSION_DIM、OUTPUT_DIM、名	与 state_dict 同步
权重路径	非“随机初始化”

表 4.3 Web–MVQA 配置对齐要点

4.4.2 功能测试（模块级与业务流程）

模块级要点（前后端）

后端接口要和表 3.1、表 2.1、/docs 对得上; 验收时重点看鉴权、上传/URL/重分析、列表、配置和敏感词, 以及 VideoResponse 别把 sessdata 漏出去。前端

用 Login、Home、FileList、ChartMain、Settings 等组件，把扫码、轮询、筛选、图表跑通。

业务流程与数据一致性

主路径：先登录 → 本地上传、批量或填 URL → 任务进队（常见先 pending，链过来的有时先 downloaded）→ 轮询详情直到终态 → 大屏看曲线；若变成 failed，界面要提示，并能重传或点 reanalyze。验收可以按表 4.4 勾（T1–T5 好自动化；T6–T7 偏手工）；和 OpenAPI 对不上的字段就算 bug。

编号	场景	预期
T1	未带令牌访问 /api/videos	401，前端清理本地令牌
T2	本地上传合法格式视频	返回 VideoResponse, status 非终态时可轮询
T3	带 token 请求 /stream	206/200，且仅能访问本人 video_id
T4	同名覆盖上传	存在 score_change 或业务定义的覆盖语义
T5	设置页保存与 checkpoint 冲突的 num_clips	推理失败或装载错误可被观察与记录
T6	覆盖 FR-6（配置、敏感词、审批、管理员 MID 等）	与表 3.1、OpenAPI 一致，手工或清单勾选
T7	覆盖 FR-7、FR-8（帮助、健康检查、静态资源、缩略图）	与表 3.1、OpenAPI 一致，手工或清单勾选

表 4.4 端到端验收场景摘录

分页列表一路「加载更多」下去，第 k 次请求带 skip_k ，这次回来 items 有 n_k 条，则应满足

$$\text{skip}_{k+1} = \text{skip}_k + n_k, \quad (4.4.1)$$

且各次加起来 $\sum_k n_k = \text{total}$ ，id 不重复；拿这个式子写脚本，能检查无限滚动有没有漏页、重页。

4.4.3 接口、性能与推理一致性

接口长什么样以 /docs 为准：skip、limit、multipart 字段名、表单和实现别各写各的。用户体感上的时延大致满足

$$T_{\text{ux}} \approx T_{\text{first}} + T_{\text{infer}} + \varepsilon, \quad (4.4.2)$$

$$0 \leq \varepsilon < T_{\text{poll}} \approx 2 \text{ s}.$$

轮询大概要 $\lceil T_{\text{infer}}/T_{\text{poll}} \rceil$ 次；顺带轻量请求 GET /health、列表接口，可粗估吞吐 $R \approx 1/\bar{T}$ 。/uploads 和带 token 的 /stream 各走各的；大文件上传别忘 axios 超时给够。

离线跑 `predict_video_qa` 和 Web 用同一套配置、同一套权重时，数值上应满足

$$\|\hat{\mathbf{y}} - \hat{\mathbf{y}}^{(\text{ref})}\|_{\infty} \leq \delta. \quad (4.4.3)$$

可在 `mvqa_analyzer._predict_video_qa_impl` 里临时 `print` 形状或 `dump` 中间结果对着看；`torch.jit` 和 `eager` 两条路容差 δ 最好分开标。cuDNN 等非确定性只要在 δ 里就算过。

非功能与安全 顺带核对式 (4.4.2)、失败有没有终态、上传路径别被穿越、JWT_SECRET 别泄露、生产 CORS 别敞太大。

4.4.4 性能与资源占用观察（建议性）

T_{infer} 大体随片长、 n_c 、 f_c 和开不开 Whisper 往上走；显存尖峰常在 ViT/AST 一批编码或长序列 LMF 那段。验收不妨记一笔：同一段素材 cold/warm 各跑多久、SQLite 文件随任务堆多大、多任务并发会不会 GPU OOM。若只能 CPU 推理，文档里写清楚最多切多少片、超时多久，别让用户以为「卡死了」。

第5章 总结与展望

5.1 全文总结

一句话概括：面向中文 UGC 短视频，在画面、声音之外，把时间对齐的弹幕和口播旁白也拉进模型里，经 LMF 和 TemporalAggregation 同时给出片段级、视频级各 12 维结果（式 (2.4.1)、表 2.2）。结构上，第 1 章先把问题和名词说清楚；第 2 章把 REST/JWT、MVQA 数据流、predict_video_qa 侧形状约束拢在一处；第 3 章把要做的事写成可追的 FR/NFR 和库表；第 4 章对照源码落地，用式 (4.4.2)、式 (4.4.1)、式 (4.4.3) 等当验收抓手；第 5 章收一收并老实讲局限。

工程上，同进程 BackgroundTasks 做异步、JWT (JWT_SECRET) 管登录、contextvars 把用户配置绑到推理链；记得让 num_clips、fusion_dim 和权重始终对得上。

5.2 创新点

模型侧：片段这一路，弹幕和音、视在 LMF 里搅在一起；整段视频这一路，先把音、视沿时间聚合，再和旁白一起进 LMF。工程侧：B 站链接拉取、扫码和 Bearer 两套鉴权、contextvars 下按用户换推理参数、敏感词审批、视频列表运维，都塞在同一套服务边界里，少出现「模型离线能跑、站点线上对不上」的漂移。

5.3 不足与未来展望

主观分还可以上公开数据集，把 SRCC/PLCC 补齐；工程上可以上任务队列、量化、单一配置源，少点漂移和并发翻车；模型上可以试多模态大模型换掉三个编码器，或把「质量」定义得更细。本文没做**模态消融**（例如关掉某一模态或换编码器）以及配套的主观/客观对比，所以「多模态比单模态好多少」主要在架构和工程闭环层面说清楚。下面按数据、工程、模型、评测四条线列一点能数得出来的后续指标，和第 4 章的工程叙述也能对上号。

数据与标注。现在主线先把系统跑稳，大规模人工打分、跨站泛化还欠着；弹幕随分区、时段差很多，冷门分区要不要单独看，以后得单聊。可以规划：固定分区抽 $N \geq 30$ 条片、每条找 K 个人标，报告 Fleiss' κ 或平均成对一致；再在公开 UGC 基准上报 PLCC/SRCC，以及跨分区掉多少（和第 2 章离线 predict_video_qa 输出对比）。

工程扩展。同进程 `BackgroundTasks` 加单文件 `SQLite`，并发写、长事务一多就会顶；若要产品化，推理最好拆到独立进程或队列，对象存储、只读副本也可以上，`uploads` 要做生命周期。验收可以量：`c` 路并发上传时 `GET /api/videos` 的 P95、WAL 下写冲突重试率、推理和 API 分进程后 GPU 是否还被 `_INFERENCE_LOCK` 卡成「单路峰值」。

模型与可解释性。12 维啥意思取决于当初怎么标；片段曲线能指到时间段，但「为啥这段分低」还可以用注意力、显著性图再画一画。可在固定片段上比 `Grad-CAM` 或跨模态注意力热图和弹幕高峰是否对得上，并报告能复现的案例有多少条。

测试层面，固定权重和种子，把 `predict_video_qa` 和 Web 落库结果按式 (4.4.3) 对齐，并记下 T_{infer} 、显存，当以后优化的基线。表 4.4 里的场景可以自动化成契约测试，省回归人力；关键路径建议每版至少跑一轮「登录—上传—轮询—得分入库」，顺便统计覆盖率和平均耗时。

参考文献

- [1] MITTAL A, MOORTHY A K, BOVIK A C. No-reference image quality assessment in the spatial domain[J]. IEEE Transactions on Image Processing, 2012, 21(12): 4695-4708.
- [2] SESHADRINATHAN K, SOUNDARARAJAN R, BOVIK A C, et al. Study of subjective and objective quality assessment of video[J]. IEEE Transactions on Image Processing, 2010, 19(6): 1427-1441.
- [3] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. 2017.
- [4] LU J, BATRA D, PARIKH D, et al. ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks[C]//Advances in Neural Information Processing Systems. 2019.
- [5] WANG Y, KE J, TALEBI H, et al. Rich features for perceptual quality assessment of UGC videos[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021: 13435-13444.
- [6] 程茹秋, 余烨, 石岱宗, 等. 图像与视频质量评价综述[J]. 中国图象图形学报, 2022, 27(5): 1410-1429.
- [7] 车万翔, 郭江, 崔一鸣. 自然语言处理: 基于预训练模型的方法[M]. 北京: 电子工业出版社, 2021.
- [8] DOSOVITSKIY A, BEYER L, KOLESNIKOV A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[C]//International Conference on Learning Representations. 2021.
- [9] GONG Y, CHUNG Y A, GLASS J. AST: Audio spectrogram transformer[C]//Proc. Interspeech. 2021: 51-55.
- [10] LIU Y, OTT M, GOYAL N, et al. RoBERTa: A robustly optimized BERT pretraining approach[C]//International Conference on Learning Representations. 2020.
- [11] RADFORD A, KIM J W, XU T, et al. Robust speech recognition via large-scale weak supervision[C]//International Conference on Machine Learning. 2023: 28492-28518.
- [12] LIU Z, SHEN Y, LAKSHMINARASIMHAN V B, et al. Efficient low-rank multimodal fusion with modality-specific factors[C]//Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Melbourne, Australia: Association for Computational Linguistics, 2018: 2247-2256.

